

Topic: Scalability Study

Doctoral Research Methodology (Course Code: CS704)

Kausthav Pratim Kalita

Department Of Computer Science And Engineering, Tezpur University

2nd Semester, PhD

Contents

1. Introduction
2. Different Types of Scalability
3. Scalability of Resources
4. Scalability of Databases
5. Weak Scalability vs. Strong Scalability
6. Scalability Testing
7. Outcomes of Scalability Testing
8. Laws Related To Scalability
9. References

Scalability is a desirable attribute of a network, system, or process. It reflects the ability of a system to accommodate an increasing number of elements or objects, to process growing volumes of work gracefully, and/or to be susceptible to enlargement [1].

The two usages of scalability are mentioned below:

1. The ability of a computer application or product (hardware or software) to function well as it (or its context) is changed in size or volume in order to meet a user need.
2. The ability not only to function well in the rescaled situation, but to actually take full advantage of it.

Introduction

A system is unscalable when the additional cost of coping with a given increase in traffic or size is excessive, or when it cannot cope with the increased level of functionality. Cost may be quantified in the following ways:

1. Response time
2. Processing overhead
3. Space
4. Memory
5. Financial hurdles

An unscalable system leads to:

1. Increase in labor cost
2. Degradation in QoS
3. Deprivation from revenue opportunities
4. Replacement of the current system with an optimized one

The ability to scale up a system or system component may depend on:

1. Data structures used
2. Algorithms used to implement the data structure used
3. The mechanisms its components use to communicate with one another

Different Types of Scalability

The different types of scalability are as follows:

1. Structural scalability
2. Load scalability
3. Space scalability
4. Space-time scalability
5. Scalability over long distances
 - 5.1 Distance Scalability
 - 5.2 Speed/Distance Scalability

Structural Scalability

Structural scalability is the ability of a system to expand in a chosen dimension without major modifications to its architecture. A system is structurally scalable if its implementation or standards do not impede the growth of the number of objects it encompasses, or at least will not do so within a chosen time frame.

Some of the scenarios where structural scalability may be hampered are:

1. A packet header field typically contains a fixed number of bits.
2. If the field is an address field, the number of addressable nodes is limited.
3. If the field is a window size, the amount of unacknowledged data is limited.
4. A telephone numbering scheme with a fixed number of digits

Load Scalability

Load scalability is the ability to function gracefully, i.e., without undue delay and without unproductive resource consumption or resource contention at light, moderate, or heavy loads while making good use of available resources. Some of the factors that can undermine load scalability include:

1. the scheduling of a shared resource
2. the scheduling of a class of resources in a manner that increases its own usage (self-expansion), and
3. inadequate exploitation of parallelism

Space Scalability

A system or application is regarded as having space scalability if its memory requirements do not grow to intolerable levels as the number of items it supports increases.

Various programming techniques might be used to achieve space scalability, such as **sparse matrix methods** [6] or **compression** [7].

Because compression takes time, it is possible that space scalability may only be achieved at the expense of load scalability.

Space-time Scalability

We regard a system as having space-time scalability if it continues to function gracefully as the number of objects it encompasses increases by orders of magnitude.

A system may be space-time scalable if the data structures and algorithms used to implement it are conducive to smooth and speedy operation whether the system is of moderate size or large.

For example, a **search engine** that is based on a linear search would not be space-time scalable, while one based on an **indexed** or **sorted data structure** such as a **hash table** or **balanced tree** could be.

Scalability Over Long Distances

Distance Scalability: An algorithm or protocol is distance scalable if it works well over long distances as well as short distances.

Speed/Distance Scalability: An algorithm or protocol is speed/distance scalable if it works well over long distances as well as short distances at high and low speeds.

Scalability of Resources

Horizontal Scaling of resources: Scaling horizontally (out/in) means adding more nodes to (or removing nodes from) a system, such as adding a new computer to a distributed software application. An example might involve scaling out from one web server to three [2].

Vertical Scaling of resources: Scaling vertically (up/down) means adding resources to (or removing resources from) a single node, typically involving the addition of CPUs, memory or storage to a single computer [2].

Scalability of Databases

Scalability for databases requires that the database system be able to perform additional work given greater hardware resources, such as additional servers, processors, memory and storage.

Algorithmic innovations have include **row-level locking** [9] and **table and index partitioning** [10]. Architectural innovations include **shared nothing** and **shared everything** architectures for managing multi-server configurations [8].

Weak Scalability vs. Strong Scalability

Weak scalability

How fast do we have to increase the problem size in order to maintain a constant efficiency when increasing the number of processors?

The slower the problem size must grow, the better the weak scalability of the system.

Strong scalability

How fast does the efficiency decrease when we increase the number of processors but keep the problem size constant?

The slower the decrease, the better the strong scalability of the system.

Scalability Testing

Scalability testing, is the testing of a software application to measure its capability to scale up or scale out in terms of any of its non-functional capability [3].

Scalability testing can give answers to the following typical questions:

1. How do hardware and software changes affect the server performance?
2. Is it possible to increase the application's productivity by upgrading the server hardware?

Creating a scalability test:

1. Incrementing loads
2. Test environment

Outcomes of Scalability Testing

Unproportional Outcome

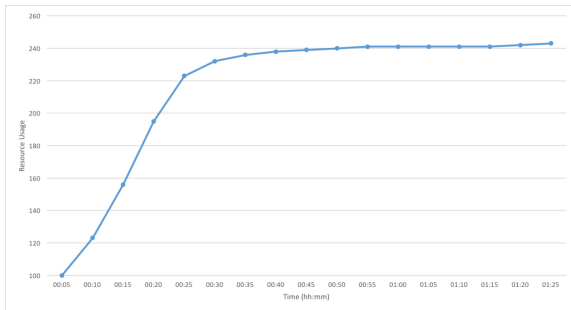


Figure 1: The graph shows resource usage (memory) over time

Outcomes of Scalability Testing

Proportional Outcome

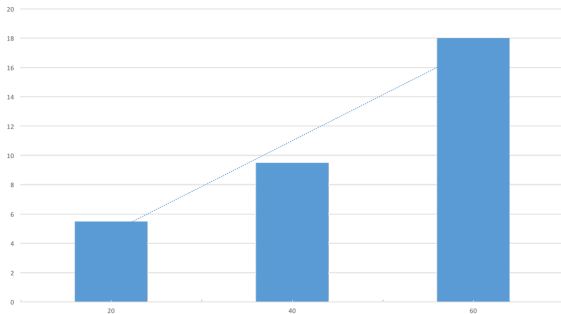


Figure 2: The graph shows resource usage (memory) over time

Laws Related To Scalability

Amdahl's law: In Amdahl's law, computational workload W is fixed while the number of processors that can work on W can be increased [4].

Denote the execution rate of i processors as R_i , then in a relative comparison they can be simplified as $R_1 = 1$ and $R_n = n$. The workload is also simplified. We assume that the workload consists of sequential work αW and n parallel work $(1 - \alpha)W$ where α is between 0 and 1.

$$S_n = \frac{n}{1 + (n - 1)\alpha}$$

Gustafson's Law: This law says that increase of problem size for large machines can retain scalability with respect to the number of processors [4].

$$S'_n = \alpha + (1 - \alpha)n$$

Laws Related To Scalability

Sun and Ni's Law: This one is referred to as a memory bound model. It turns out that when the speedup is computed by the problem size limited by the available memory in n-processor system, it leads to a generalization of Amdahl's and Gustafson's law [4].

For n nodes, assume the parallel portion of workload is increased by $G(n)$ times reflecting the increase of memory in n-node system, i.e.,

$$W^* = \alpha W + (1 - \alpha)G(n)W$$

Generalized equation:

$$S^* = \frac{\alpha + (1 - \alpha)G(n)}{\alpha + (1 - \alpha)G(n)/n}$$

Here,

$G(n) = 1$ gives Amdahl's Law.

$G(n) = n$ gives Gustafson's Law.

1. Bondi AB. Characteristics of scalability and their impact on performance. In Proceedings of the 2nd international workshop on Software and performance 2000 Sep 1 (pp. 195-203). ACM.
2. Scalability. Available Online: <https://en.wikipedia.org/wiki/Scalability> (accessed on 12 November 2019)
3. What Is Scalability Testing? How To Test The Scalability Of An Application. Available Online: <https://www.softwaretestinghelp.com/what-is-scalability-testing/> (accessed on 12 November 2019)
4. Multiprocessor Laws. Available Online: <http://www.d.umn.edu/~tkwon/course/5315/HW/MultiprocessorLaws.pdf> (accessed on 12 November 2019)

5. Scalability. Available Online: <https://www.comp.nus.edu.sg/seer/book/2e/Ch06.%20Scalability.pdf> (accessed on 12 November 2019)
6. Gupta A, Karypis G, Kumar V. Highly scalable parallel algorithms for sparse matrix factorization. IEEE Transactions on Parallel and Distributed Systems. 1997 May;8(5):502-20.
7. Plaza A, Plaza J, Paz A. Improving the scalability of parallel algorithms for hyperspectral image analysis using adaptive message compression. In 2009 IEEE International Geoscience and Remote Sensing Symposium 2009 Jul 12 (Vol. 4, pp. IV-196). IEEE.
8. Shared Nothing vs Shared Everything. Available Online: <http://kamalmeet.com/dbms/shared-nothing-vs-shared-everything/> (accessed on 12 November 2019)

9. Row Level Locking. Available Online: <https://riptutorial.com/mysql/example/24166/row-level-locking> (accessed on 12 November 2019)
10. Using Table and Index Partitioning. Available Online: <https://docs.microsoft.com/en-us/sql/relational-databases/server-management-objects-smo/tasks/using-table-and-index-partitioning?view=sql-server-ver15> (accessed on 12 November 2019)

THANK YOU